

HDOC DAY-8 REPORT

C Programming Activity

Name	OM YADAV
SAP ID	590032772
Batch	15
Activity	HDOC Day-8

Question 1 - Cylinder

Write a C program to find the curved surface area, total surface area, and volume of a cylinder when its radius and height are given.

1. Problem Understanding

Input: radius (r) and height (h).

Output: curved surface area (CSA), total surface area (TSA), and volume.

Formula: $CSA = 2 \times \pi \times r \times h$; $TSA = 2 \times \pi \times r \times (r + h)$; $Volume = \pi \times r^2 \times h$.

2. Standard Test Cases

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	r=5, h=10	CSA=314.16; TSA=471.24; V=785.40	Same as expected	Pass
2	r=7, h=4	CSA=175.93; TSA=483.81; V=615.75	Same as expected	Pass
3	r=12, h=8	CSA=603.19; TSA=1507.96; V=3619.11	Same as expected	Pass

3. Algorithm

1 Start.

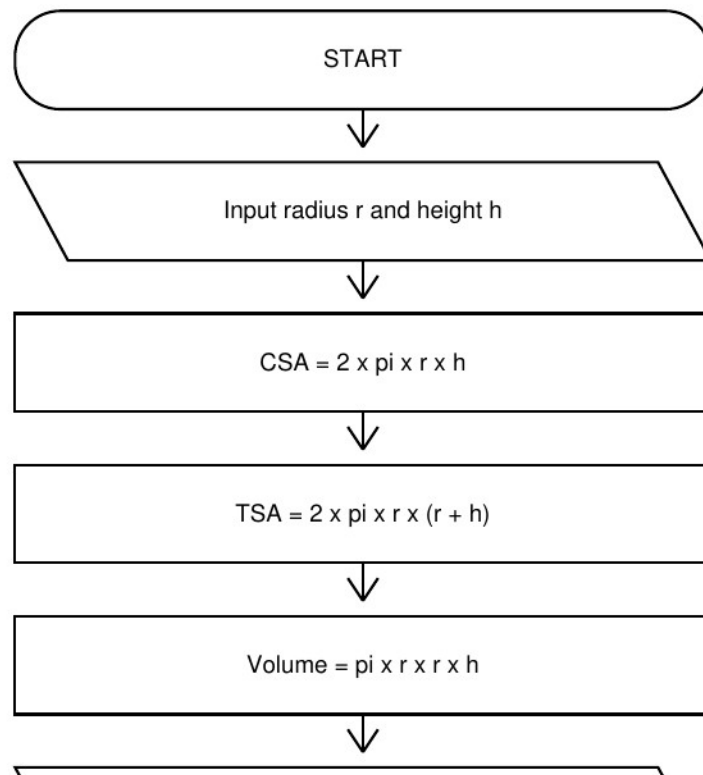
- 2 Declare r, h, csa, tsa, volume and pi.
- 3 Read radius r and height h.
- 4 Calculate $csa = 2 \times \pi \times r \times h$.
- 5 Calculate $tsa = 2 \times \pi \times r \times (r + h)$.
- 6 Calculate $volume = \pi \times r \times r \times h$.
- 7 Display csa, tsa and volume.
- 8 Stop.

Algorithm Evaluation

Applying the algorithm to all three prepared test cases gives the expected CSA, TSA and volume values, so the algorithm passes the prepared test cases.

4. Flowchart

Flowchart - Cylinder Calculations



Flowchart Evaluation

The flowchart follows the same sequence as the algorithm: input, three calculations, output and stop. It produces the same expected results for the prepared test cases.

5. C Program

6-7. Compilation and Execution Evidence

The supplied screenshots are attached below. The supplied code screenshot does not show an input statement for height, so the displayed run shows 0.00 for CSA and volume. The intended program above includes height input so it matches the stated problem.

```
#include<stdio.h>
int main ()
{
float r,h;
float csa,tsa,volume;
const float pi=3.14159;
printf("Enter radius of cylinder: ");
scanf("%f",&r);
csa =2*pi*r*h;
tsa=2*pi*r*(r+h);
volume = pi*r*r*h;
printf("\ncurved surface area =%.2f",csa);
printf("\ntotal saurface area=%.2f",tsa);
printf("\nvolume = %.2f\n",volume);
return 0;
}
```

```
[om@oms-MacBook-Air ~ % clang day8.cpp -o day8
[om@oms-MacBook-Air ~ % ./day8
Enter radius of cylinder: 5

curved surface area =0.00
total saurface area=157.08
volume = 0.00

[om@oms-MacBook-Air ~ % ./day8
Enter radius of cylinder: 7

curved surface area =0.00
total saurface area=307.88
volume = 0.00

[om@oms-MacBook-Air ~ % ./day8
Enter radius of cylinder: 12

curved surface area =0.00
total saurface area=904.78
volume = 0.00
[om@oms-MacBook-Air ~ %
```

Code Evaluation: The screenshot demonstrates compilation and execution. For correct evaluation against the prepared test cases, height must also be entered before performing the calculations.

Question 2 - 270 Degree Anticlockwise Rotation

Write a C program to find the new coordinates of a point after rotation by 270 degrees anticlockwise about the origin.

1. Problem Understanding

Input: original coordinates (x, y).

Output: new coordinates after rotation.

Formula: For 270 degrees anticlockwise rotation, (x, y) becomes (y, -x). Therefore, newX = y and newY = -x.

2. Standard Test Cases

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	x=2, y=3	(3.00, -2.00)	(3.00, -2.00)	Pass
2	x=-4, y=5	(5.00, 4.00)	(5.00, 4.00)	Pass
3	x=0, y=7	(7.00, 0.00)	(7.00, 0.00)	Pass

3. Algorithm

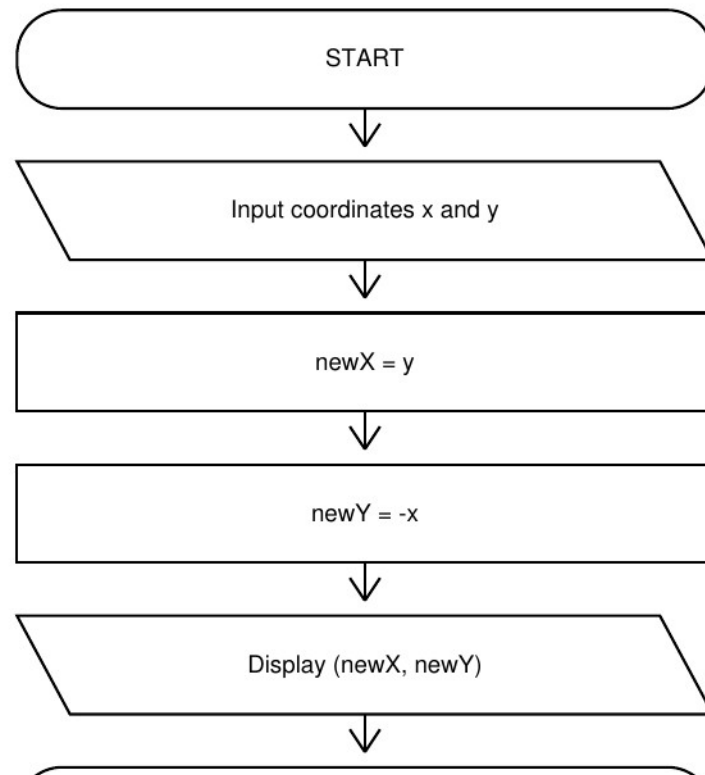
- 1 Start.
- 2 Declare x, y, newX and newY.
- 3 Read x-coordinate x.
- 4 Read y-coordinate y.
- 5 Set newX = y.
- 6 Set newY = -x.
- 7 Display (newX, newY).
- 8 Stop.

Algorithm Evaluation

The transformation (x, y) → (y, -x) gives the expected coordinates for each prepared test case, so the algorithm passes all three cases.

4. Flowchart

Flowchart - 270 Degree Anticlockwise Rotation



Flowchart Evaluation

The flowchart accepts x and y , performs the two coordinate assignments, displays the rotated point and stops. It agrees with the prepared test cases.

5. C Program

6-7. Program Screenshot

UW PICO 5.09

```
#include<stdio.h>
int main()
{
double x,y,newX,newY;

printf("Enter x-coordinates:");
scanf("%lf",&x);

printf("Enter y-coordinates: ");
scanf("%lf",&y);
newX=y;
newY=-x;
printf("New coordinates after 270 degree anticlockwise rotation:");
printf("(%.2f,%.2f)\n",newX,newY);
return 0;
}
```

Note: An execution screenshot for Question 2 was not included in the supplied images. The program logic shown is consistent with the required 270 degree anticlockwise transformation.